Wprowadzenie do C++

zmienna

1. zmienna ma swoją nazwę (dla programisty) i obszar w pamięci
2. W pamięci znajdują się bity
3. Typ zmiennej określa jak interpretowane są bity w pamieci powiązanej ze zmienną

bool

1. 1 bit
2. wartość true lub false
3. sizeof()

liczby całkowite

1. short, int, long itp.
2. signed i unsigned
3. uint8_t, int32 itd.

typy zmiennoprzecinkowe

1. float
2. double
3. long double

znaki

1. char
2. static_cast<int>(znak)

ciąg znaków

1. string
2. #include <string>

instrukcje warunkowe

```
if (x > 0) { … }
else if (x == 0) { … }
else { … };
```

zadanie!

Napisz swój pierwszy prosty kalkulator! Zadeklaruj zmienną *operator* typu znakowego i zmienne liczbowe *x* i *y*. Wczytaj z klawiatury wartości zmiennych liczbowych i operator (+, -, *, /). Zależnie od wprowadzonego operatora wykonaj odpowiednie działanie na x i y i zwróć jego wynik.

instrukcja switch

```
switch (x) {
    case 1:
        ...
        break;
    case 2:
        ...
        break;
    default: ...
}
```

zadanie!

Stwórz kalkulator wyliczający cenę biletów uwzględniając wszystkie poniższe zasady:

- Cena bazowa biletu to 80zł
- Dzieci do 6 roku życia wchodzą za darmo
- Zniżka -30 zł w każdą środę
- Zniżki nie łączą się, w przypadku dwóch dostępnych zniżek wybierz korzystniejszą dla klienta.
- Zniżka -30% do 26. roku życia
- W weekendy +15 zł do ceny bazowej
- Zniżka -50% dla seniorów powyżej 60. roku życia
- Zniżka dla grup powyżej 10 osób -15%

Wypisz cenę pojedynczego biletu dla: studenta w środę, grupy 12 seniorów w sobotę, dorosłego w niedzielę, grupy 30 uczniów we wtorek

na następnej lekcji sprawdzian!

typy danych, if, switch

pętla for

pętla wykonuje polecenia do czasu spełnienia warunku zakończenia pętli

1. inicjalizacja zmiennej
2. warunek zakończenia pętli
3. modyfikacja wartości zmiennej
4. i może zostać użyte jako zmienna w środku pętli

```
for (int i = 0; i < 5; i++) {
    cout << i << endl;
}
```

pętla for - break

pętlę można przerwać za pomocą polecenia break

```
for (int i = 0; i < 5; i++) {
    if (i == 5) {
        break;
    }
    cout << i << endl;
}
```

zadanie!

1. Za pomocą pętli for wypisz na ekranie wszystkie liczby parzyste od 1 do x (int, przykładowo x=100).

2. Napisz program, w którym masz dokładnie 4 próby na wpisanie poprawnego PINu. Zadeklaruj w zmiennej int pin (np. 1234) i daj użytownikowi 4 próby, po których program się kończy z informacją, że urządzenie zostało zablokowane. Jeśli użytkownik wpisze prawidłowy pin wcześniej, wyświetl komunikat o pomyślnym odblokowaniu i przerwij pętlę.

pętla while

1. sprawdzenie warunku
2. wykonanie instrukcji

```
int i = 1;
while (i <= 5) {
    cout << "Liczba: " << i << endl;
    i++;
 }
```

while(true)

```
while (true) {
    cout << "Pętla nieskończona!!!" << endl;
 }
```

zadanie!

1. Napisz program symulujący skarbonkę, który prosi użytkownika o podanie kwoty którą chce uzbierać (np. 100 zł) i kwot kolejnych wpłat, które są dodawane do sumy. Kiedy suma zrówna się lub przekroczy cel, program wyświetla komunikat “Udało ci się uzbierać cel!”

rand()

Funkcja do generowania liczb pseudolosowych.
Wygeneruj liczbę pomiędzy 1 a 100:

```
#include <cstdlib>
int  randomNum = rand() % 100;
```

zadanie!

1. Napisz prostą grę konsolową w zgadywanie liczb. Na początku gry program generuje pseudolosową liczbę całkowitą X z zakresu 1-200, którą użytkownik próbuje zgadnąć. Po każdej próbie program wyświetla, czy podana przez użytkownika liczba jest mniejsza czy większa od X. Gra kończy się kiedy użytkownik zgadnie X, wtedy wyświetlany jest komunikat gratulacje zgadłeś.

lepsza metoda - biblioteka <random>

```
#include <random>
using namespace std;

int main() {
    random_device rd;
    mt19937 gen(rd());  // setup
    uniform_int_distribution<int> distrib(-50, 50);
}

cout << distrib(gen);
```

zadanie!

Napisz program, który symuluje pomiary temperatury na zewnątrz. Załóż ilość pomiarów n=10 i n razy wygeneruj pseudolosowy odczyt temperatury z zakresu od -20 stopni do 36 stopni (random int, np. 10 stopni, 15 stopni, -5 stopni). Po zakończeniu pomiarów zwróć średnią temperaturę, najniższą i najwyższą zanotowaną.

tablice (array)

Zmienna przechowująca zbiór elementów tego samego typu. Deklarujemy typ elementów tablicy, opcjonalnie jej wielkość.

```
int oceny[5];
int liczby[5] = {2, 4, 6, 8, 10};
int dane[] = {10, 20, 30};
```

indeksowanie tablicy

Każdy element tablicy ma swój indeks zaczynając od 0.

```
cout << liczby[0];
liczby[1] = 99;
```

pętla for po tablicy

Przykład przejścia pętlą for po elementach tablicy:

```
for (int i = 0; i < 5; i++)
{
    cout << liczby[i] << endl;
}
```

zadanie!

Utwórz tablicę ocen ucznia zawierajacą wartości [5, 4.5, 3.75, 1, 3.5, 5]. Oblicz średnią ocenę za pomocą pętli for po tablicy.

tablice dwuwymiarowe

Tablica, która zawiera wiersze i kolumny.

int tab[3][3];

[0][0]	[0][1]	[0][2]
[1][0]	[1][1]	[1][2]
[2][0]	[2][1]	[2][2]

deklaracja tablicy

```
int tab[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};

tab[0][1] = 99;
```

pętla po tablicy dwuwymiarowej

```
for (int i = 0; i < 2; i++)
{
    for (int j = 0; j < 3; j++) {
        cout << tab[i][j] << " ";
    }
    cout << endl;
}
```

zadanie!

Utwórz tablicę wyników rzutu piłką lekarską, dla trzech uczniów, każdy po 3 próby (wartości z zakresu 6-12[m], wymyślone lub wylosowane).

- Wyświetl wyniki w postaci tabeli, po jednym wierszu na ucznia
- Zmień wynik pierwszego ucznia z pierwszego przedmiotu na 11.6
- Ponownie wyświetl cała tabelę

funkcje

Funkcję stosujemy:

- by nie powtarzać kodu,
- by dzielić program na mniejsze częsci związane z jakąś funkcjonalnością

```
int main() {
    ...
}
```

typ zwracany z funkcji

Funkcja zwracjąca zmienną typu int:

```
int zwieksz(int x) {
    return x+10;
}
```

Funkcja, która nic nie zwraca:

```
void zwieksz(int x) {
    x = x + 10;
}
```

wywołanie funkcji

```
int dodaj(int a, int b) {
    return a + b;
}

int main() {
    int x = 5;
    int y = 10;

    dodaj(x, y);
    dodaj(5, 10);
    dodaj(x, 20);
    dodaj(x + 2, y * 3);
}
```

przekazywanie wartości do funkcji

deklaracja funkcji z parametrami a i b

wywołanie funkcji z róznymi argumentami

przekazywanie wartości lub referencji

```
void podwoj(int &x) {
    x = x * 2;
}

int main() {
    int liczba = 5;
    podwoj(liczba);
}
```

przekazywanie referencji do funkcji

funkcja zmienia wartość zmiennej globalnie

argumenty z domyślnymi wartościami

```
void powitaj(string imie, string powitanie = "Cześć") {
    cout << powitanie << ", " << imie << "!" << endl;
}

int main() {
    powitaj("Adam");
    powitaj("Adam", "Dzień dobry");
}
```

parametr nieobowiązkowy zawsze na końcu

przy wywołaniu z jednym argumentem przyjmowana jest wartość domyślna

zadanie!

Napisz funkcję która zwraca true jeśli podana liczba typu int jest pierwsza (jest ona większa od 1 i nie posiada żadnych dzielników poza 1 oraz samą sobą). Sprawdź za pomocą tej funkcji i wypisz wszystkie liczby pierwsze od 1 do 100.

zadanie!

Napisz program symulujący wpłacanie i wypłacanie pieniędzy z konta bankowego.

1. Utwórz zmienną double saldo = 1000
2. Napisz funkcję **wplac**, która przyjmuje saldo konta oraz kwotę wpłaty i zwiększa saldo o podaną kwotę
3. Napisz funkcję **wyplac**, która przyjmuje saldo konta oraz kwotę wypłaty i zmniejsza saldo o podaną kwotę
4. W funkcji main() pobieraj w pętli od użytkownika kwoty wpłaty lub wypłaty, wywołaj odpowiednie funkcje i wyświetl końcowe saldo

przekazywanie tablicy do funkcji

- tablice są domyśnie przekazywane do funkcji jako wskazanie w pamięci pierwszego elementu tablicy
- oprócz wskażnika na pierwszy element można przekazać do funkcji również rozmiar tablicy

typ funkcji | nazwa funkcji | przekazana tablica | rozmiar tablicy

```
void wypiszTablice(int tab[], int rozmiar) {
    for (int i = 0; i < rozmiar; i++) {
        cout << tab[i] << " ";
    }
}
```

wywołanie funkcji

```
int main() {
    int liczby[4] = {10, 20, 30, 40};
    wypiszTablice(liczby, 4);
}
```

przekazana tablica

rozmiar tablicy

modyfikacja tablicy wewnątrz funkcji

Przekazanie przez referencje oznacza, że funkcja operuje na oryginale, każda zmiana elementu tablicy w środku funkcji zmieni ten element globalnie

```
void zerujTablice(int tab[], int rozmiar) {
    for (int i = 0; i < rozmiar; i++) {
        tab[i] = 0;
    }
}
```

zadanie!

1. Zadeklaruj tablicę 5 liczb całkowitych i wypełnij ją dowolnymi wartościami punktów z testu (0-100).
2. Napisz funkcję **obliczSrednia**, która przyjmuje tablicę oraz jej rozmiar, a następnie za pomocą pętli oblicza i zwraca (jako double) średnią z tych wartości.
3. Napisz funkcję **dodajBonus**, która przyjmuje tablicę i jej rozmiar, a następnie dodaje do każdego elementu w tablicy 5 punktów (zmieniając oryginał).
4. Wypisz na ekranie średnią za pomocą pierwszej funkcji, następnie wywołaj drugą funkcję i pętlą for wypisz zaktualizowane wyniki z tablicy.

zadanie*!

- Zadeklaruj tablicę plansza[3][3] i wypełnij ją znakami 'o'. Na samym środku (indeks [1][1]) wstaw znak 'x' reprezentujący gracza.
- Napisz funkcję **wypiszPlansze**(char plansza[][3]), która za pomocą zagnieżdżonych pętli wyświetli siatkę gry w konsoli.
- Napisz funkcję **przesun**(char plansza[][3], char kierunek). Funkcja powinna najpierw przeszukać tablicę, by znaleźć obecne współrzędne 'x', a następnie (zależnie od przekazanego znaku kierunku: w, s, a, d) zamienić go miejscami z sąsiadującym 'o'.
- Zabezpiecz aby 'x' nie mógł "wyjść" poza krawędzie planszy!
- W pętli wyświetlaj planszę i pobieraj od użytkownika kierunek ruchu

wskaźniki

Każda zmienna w pamięci RAM ma swój unikalny adres (zapisywany zazwyczaj w systemie szesnastkowym, np. 0x61ff08). Wskaźnik to specjalna zmienna, która nie przechowuje zwykłej **wartości** (np. 5), tylko **adres w pamięci**. Typ wskaźnika musi zgadzać się z typem zmiennej, na którą wskazuje.

```
int liczba = 42;
cout << liczba << endl;
cout << &liczba << endl;
```

wypisana wartość 42

wypisany adres w pamięci

deklaracja wskaźnika

Gwiazdka * przy deklaracji oznacza, że tworzymy wskaźnik, a nie zwykłą zmienną.

```
int wiek = 18;
int* wsk = &wiek;

cout << wsk << endl;
cout << *wsk << endl;

*wsk = 25;
cout << wiek << endl;
```

wskaźnik przechowuje adres zmiennej wiek

wypisze adres

wypisze wartość spod adresu

zmiana wartości pod adresem

wskaźniki w funkcji

Zamiast referencji (&) do funkcji możemy przekazać adres zmiennej, a w parametrze funkcji odebrać go jako wskaźnik.

```
void zwieksz(int* x) {
    *x = *x + 10;
}

int main() {
    int punkty = 50;
    zwieksz(&punkty);
    cout << punkty << endl;
}
```

zadanie!

Napisz program sterujący zużyciem energii w urządzeniu.

1. Zadeklaruj dwie zmienne całkowite: bateriaGlowna = 50 oraz bateriaZapasowa = 100, a także jeden wskaźnik int* aktywnaBateria.
2. Napisz funkcję zuzyjEnergie(int ile), która zmniejsza poziom energii o podaną wartość (pilnując, aby poziom energii nie spadł poniżej 0).
3. Napisz funkcję laduj(int* bateria, int ile), która zwiększa poziom energii pod wskazanym adresem o podaną wartość (maksymalnie do 100).
4. Jeśli funkcja zuzyjEnergie pobierze wiecej energii niż jest w baterii glownej, pobierz resztę z baterii zapasowej i zamień je w pamięci (podmień adres głównej i zapasowej). Po każdej operacji wypisz aktualny stan obu baterii.